UNIVERSIDAD AUTÓNOMA DE CHIRIQUÍ
VICERRECTORÍA ACADÉMICA
FACULTAD DE ECONOMÍA
ESCUELA DE CIENCIAS COMPUTACIONALES

Nombre de la asignatura			Abreviatura	Código	Año / Semestre
PROGRAMACIÓN V			**PROG 309**	**28186**	**2026 / II**
Nombre del docente			**Correo Electrónico**		
ALEXIS FLORES FRANCO			**alexis.flores@unachi.ac.pa**		
Días en que se imparte	**Jornada**	**Horario**	**Sesiones / Horas**	**H.T / H.L**	**Créditos**
LUNES – JUEVES – VIERNES	**MATUTINA**	Lunes 9:35-10:20 (Teoría) Jueves 7:00-8:35 (Lab.) Viernes 7:00-8:35 (Lab.)	SESIONES: 80 HORAS: 80	**H.T: 1 H.L: 4**	**3**
Fecha de inicio de la asignatura			**Fecha de culminación de la asignatura**		
17 – Agosto – 2026			**4 – Diciembre – 2026**		
Fecha de revisión por el Director(a) del Departamento		**Firma**	**Fecha de entrega al estudiante**		

Compromisos del Estudiante

- Trabajar en equipo, con honestidad y responsabilidad.
- Comprometerse a realizar las actividades programadas.
- Dedicación, tiempo y pasión en cada una de las actividades.
- Realizar un seguimiento constante de las actividades.
- Asistir puntualmente a las sesiones presenciales y entregar las asignaciones en el tiempo planificado.
- Solicitar apoyo u orientación en el momento que lo requiera, considerando los horarios de atención.
- Respetar el espacio de trabajo en clases, participando cuando se requiera.

Entiendo y acepto que durante cada sesión configuraré mi celular en modo silencioso, no lo utilizaré en el aula de clases, no comeré en el aula, seguiré las indicaciones que se informen, tendré una buena presentación personal y cumpliré los acuerdos establecidos.

Evaluación (Art. 166 Estatuto Universitario)
Tareas, investigaciones, talleres y otras 35%
Exámenes parciales (mínimo 2) 30%
Examen semestral 35%

PLANIFICACIÓN DIDÁCTICA

EJE TEMÁTICO	N° 1	Introducción a los Patrones Arquitecturales
COMPETENCIAS GENÉRICAS	● Capacidad de abstracción, análisis y síntesis. ● Capacidad de aplicar los conocimientos en la práctica. ● Capacidad para identificar, plantear y resolver problemas. ● Capacidad de trabajo en equipo.	
COMPETENCIAS ESPECÍFICAS	● Reconoce e infiere sobre la importancia de los patrones de diseño arquitecturales. ● Identifica las ventajas y desventajas de los patrones de diseño arquitecturales.	

FECHA	CONTENIDOS	ACTIVIDADES ACADÉMICAS INNOVADORAS APLICADAS POR EL DOCENTE
17 Agosto – 11 septiembre. Semana 1	• ¿Qué es un patrón arquitectónico? Concepto y utilidad en la ingeniería de software. • Patrón de capas (introducción). • Patrón cliente-servidor (introducción).	**Clase Teórica** 1. Bienvenida y auto-presentación (docente). 2. Presentación del programa, ejes temáticos y forma de evaluación del semestre. 3. Introducción a los patrones de arquitectura de software. **Actividad Innovadora: Lluvia de ideas** **Desafío cognitivo:** *¿Por qué no todas las aplicaciones de software deberían construirse con la misma estructura interna?* **Laboratorio** **Preparación del entorno de desarrollo** Actividades: • Instalar y configurar el IDE, el lenguaje de programación y el sistema de control de versiones (Git) que se usarán durante el semestre. • Crear un repositorio de trabajo personal para las prácticas del curso. • Elaborar un diagrama esquemático simple que represente un patrón de capas conocido por el estudiante (ej. una app de su celular).
Semana 2	• Patrón maestro-esclavo (master-slave). • Patrón de filtro de tubería (pipe-filter).	**Clase Teórica** **Actividad Innovadora: Clase demostrativa** **Desafío cognitivo:** *¿Qué tipo de aplicaciones se beneficiarían de dividir el procesamiento en una tubería de filtros independientes?* **Laboratorio** **Modelado de patrones maestro-esclavo y pipe-filter** Actividades: • Diagramar, en equipos, un caso de uso real que aplique el patrón maestro-esclavo (ej. procesamiento distribuido de tareas). • Diagramar un caso de uso que aplique el patrón de filtro de tubería (ej. procesamiento de una imagen en varias etapas). • Presentar ambos diagramas al grupo y justificar la elección del patrón.
Semana 3	• Patrón de intermediario (broker). • Patrón de igual a igual (peer-to-peer). • Patrón de bus de eventos (event-bus).	**Clase Teórica** **Actividad Innovadora: Conversatorio académico** **Desafío cognitivo:** *¿Qué ventajas ofrece un patrón de bus de eventos frente a la comunicación directa entre módulos de un sistema?* **Laboratorio** **Investigación aplicada de patrones de comunicación** Actividades: • Investigar, por equipos, un caso real de la industria que utilice el patrón broker, P2P o bus de eventos. • Elaborar un diagrama de componentes que muestre la comunicación entre los elementos del patrón investigado.

		• Compartir los hallazgos en un conversatorio grupal.
Semana 4	• Patrón modelo-vista-controlador (MVC) (introducción). • Patrón de pizarra (blackboard). • Patrón de intérprete.	**Clase Teórica** **Actividad Innovadora: Taller comparativo** **Desafío cognitivo:** *Dado un sistema de reservas de vuelos, ¿cuál de los diez patrones estudiados sería el más adecuado para su arquitectura? Justifique.* **Laboratorio** **Selección de patrón arquitectónico para un caso dado** Actividades: • Analizar un caso de estudio proporcionado por el docente (requerimientos de un sistema ficticio). • Seleccionar, en equipo, el patrón arquitectónico más adecuado entre los diez estudiados y justificar la decisión por escrito. • Elaborar un diagrama de alto nivel de la arquitectura propuesta. **Evaluación: Cuestionario corto sobre los patrones arquitecturales estudiados.**

ESTRATEGIAS DE APRENDIZAJE (las que utilizará el estudiante)	**ACTIVIDADES ACADÉMICAS DE INVESTIGACIÓN Y EXTENSIÓN**
• Exposiciones dialogadas sobre los conceptos de patrones de arquitectura de software. • Clases demostrativas con ejemplos reales de patrones maestro-esclavo y pipe-filter. • Conversatorios académicos sobre patrones de comunicación entre componentes. • Talleres comparativos para la selección justificada de un patrón arquitectónico.	• Los estudiantes investigan un caso real de la industria que aplique uno de los patrones estudiados y presentan un informe breve. **De extensión:** • Elaboración de un mapa conceptual comparativo de los diez patrones arquitecturales, para compartir con la comunidad estudiantil.
TÉCNICA O ESTRATEGIA DE EVALUACIÓN	**RECURSOS Y TECNOLOGÍA APLICADA EN EL PROCESO DE ENSEÑANZA Y APRENDIZAJE**
• Diagramas de arquitectura elaborados en cada laboratorio. • Participación en los conversatorios y talleres. • Cuestionario corto sobre los diez patrones arquitecturales introducidos.	**Documentación oficial:** Documentación oficial del lenguaje y framework seleccionados para el curso. Guías de patrones de arquitectura de software (ver fuentes bibliográficas). **Software y herramientas:** IDE de desarrollo (ej. Visual Studio Code). Sistema de control de versiones (Git/GitHub). Herramienta de diagramación (ej. draw.io). **Recursos didácticos:** Presentaciones en diapositivas. Pizarra digital interactiva. Plataforma virtual institucional (LMS).
FUENTES BIBLIOGRÁFICAS	
Huaman, W. C. (2018). Los 10 patrones comunes de arquitectura de software. Medium. **Blancarte, O. (2020). Introducción a la arquitectura de software.**	

PLANIFICACIÓN DIDÁCTICA

EJE TEMÁTICO	N° 2	Patrón de Capas
COMPETENCIAS GENÉRICAS	• Capacidad de aplicar los conocimientos en la práctica. • Capacidad para organizar y planificar el tiempo. • Habilidades en el uso de las TIC.	
COMPETENCIAS ESPECÍFICAS	• Aplica el software requerido para el desarrollo de patrones de diseño arquitecturales. • Implementa patrones de diseño arquitecturales.	
FECHA	**CONTENIDOS**	**ACTIVIDADES ACADÉMICAS INNOVADORAS APLICADAS POR EL DOCENTE**
14 Septiembre – 9 octubre. Semana 5	• Requerimientos del escenario: definición de un caso de estudio (ej. sistema de gestión de inventario).	**Clase Teórica** **Actividad Innovadora: Estudio de caso** **Desafío cognitivo:** *¿Qué problemas podría causar mezclar la lógica de negocio con el acceso a datos en una sola capa?* **Laboratorio** **Definición del escenario y los requerimientos** Actividades: • Analizar el caso de estudio asignado por el docente para el proyecto del eje. • Redactar la descripción del escenario: actores, objetivos y alcance del sistema a construir con patrón de capas. • Identificar, en equipo, las capas necesarias para el sistema (presentación, negocio, datos).
Semana 6	• Requisitos del software: requisitos funcionales y no funcionales.	**Clase Teórica** **Actividad Innovadora: Taller** **Desafío cognitivo:** *¿Cómo afecta un requisito no funcional (ej. rendimiento) a la forma en que se diseña una capa de datos?* **Laboratorio** **Especificación de requisitos de software** Actividades: • Redactar los requisitos funcionales y no funcionales del sistema en capas del proyecto del eje. • Seleccionar el lenguaje de programación y las herramientas/framework a utilizar para la implementación. • Documentar los requisitos en una plantilla de especificación de software (SRS simplificado).
Semana 7	• Análisis y diseño de requisitos: diagrama de la arquitectura en capas.	**Clase Teórica** **Actividad Innovadora: Clase demostrativa** **Desafío cognitivo:** *¿Qué información debería fluir de la capa de presentación hacia la capa de negocio, y qué no debería fluir en sentido contrario?* **Laboratorio** **Diseño de la arquitectura en capas** Actividades:

		• Elaborar el diagrama de arquitectura en capas del proyecto (presentación, negocio, acceso a datos). • Definir las interfaces de comunicación entre cada capa. • Validar el diseño con el docente antes de iniciar la implementación.
Semana 8	• Implementación de la aplicación con patrón de capas.	**Clase Teórica** **Actividad Innovadora: Aprendizaje basado en proyecto** **Desafío cognitivo:** *Durante la implementación, ¿qué señales indicarían que una capa está asumiendo responsabilidades que no le corresponden?* **Laboratorio** **Implementación del patrón de capas** Actividades: • Codificar la capa de acceso a datos del proyecto (conexión, consultas básicas). • Codificar la capa de negocio (reglas y lógica de la aplicación). • Codificar la capa de presentación e integrar las tres capas, verificando su funcionamiento. **Evaluación: Parcial No. 1**

ESTRATEGIAS DE APRENDIZAJE (las que utilizará el estudiante)	**ACTIVIDADES ACADÉMICAS DE INVESTIGACIÓN Y EXTENSIÓN**
• Estudio de caso para la definición del escenario del proyecto. • Talleres para la especificación de requisitos de software. • Clases demostrativas del diseño de arquitecturas en capas. • Aprendizaje basado en proyecto durante la implementación del patrón.	• Investigación sobre buenas prácticas de separación de responsabilidades entre capas en proyectos reales. **De extensión:** • Elaboración de una guía breve sobre patrón de capas para compartir con estudiantes de cursos previos de programación.
TÉCNICA O ESTRATEGIA DE EVALUACIÓN	**RECURSOS Y TECNOLOGÍA APLICADA EN EL PROCESO DE ENSEÑANZA Y APRENDIZAJE**
• Documento de requerimientos y especificación de requisitos del escenario. • Diagrama de arquitectura en capas evaluado por el docente. • Evaluación práctica de la implementación funcional del patrón de capas. • Parcial No. 1 sobre patrones arquitecturales y patrón de capas.	**Documentación oficial:** Documentación oficial del lenguaje/framework seleccionado. Guías de patrones de diseño en capas. **Software y herramientas:** IDE de desarrollo. Motor de base de datos (ej. MySQL/PostgreSQL/SQLite). Herramienta de diagramación UML. **Recursos didácticos:** Plantilla de especificación de requisitos (SRS). Diagramas de arquitectura de referencia. Plataforma virtual institucional.

FUENTES BIBLIOGRÁFICAS

Conesa Caralt, J. (2010). Introducción a .NET. Editorial UOC.

Nossa Ortiz, L., & Contreras Castro, M. D. (2020). JavaScript. Editorial Académica Española.

PLANIFICACIÓN DIDÁCTICA

EJE TEMÁTICO	N° 3	Patrón de Cliente/Servidor
COMPETENCIAS GENÉRICAS	• Capacidad de aplicar los conocimientos en la práctica. • Capacidad de investigación. • Capacidad para tomar decisiones.	
COMPETENCIAS ESPECÍFICAS	• Aplica el software requerido para el desarrollo de patrones de diseño arquitecturales. • Implementa patrones de diseño arquitecturales.	
FECHA	**CONTENIDOS**	**ACTIVIDADES ACADÉMICAS INNOVADORAS APLICADAS POR EL DOCENTE**
12 Octubre – 6 noviembre. Semana 9	• Requerimientos del escenario: definición de un caso cliente-servidor (ej. API REST de un sistema de reservas).	**Clase Teórica** **Actividad Innovadora: Lluvia de ideas** **Desafío cognitivo:** *¿Qué ocurre si el servidor de una aplicación cliente-servidor se satura de solicitudes simultáneas?* **Laboratorio** **Definición del escenario cliente-servidor** Actividades: • Analizar el nuevo caso de estudio para el proyecto de este eje (arquitectura cliente-servidor). • Identificar los actores del sistema: los distintos tipos de clientes y las responsabilidades del servidor. • Definir el protocolo de comunicación a utilizar (ej. HTTP/REST) entre cliente y servidor.
Semana 10	• Requisitos del software para la arquitectura cliente-servidor.	**Clase Teórica** **Actividad Innovadora: Taller** **Desafío cognitivo:** *¿Qué requisitos no funcionales son especialmente críticos en una arquitectura cliente-servidor (disponibilidad, latencia, seguridad)?* **Laboratorio** **Especificación de requisitos del sistema cliente-servidor** Actividades: • Redactar los requisitos funcionales y no funcionales del sistema cliente-servidor del proyecto. • Seleccionar el framework o librería para implementar el servidor (ej. Express, Flask, Spring Boot). • Documentar los endpoints o servicios que expondrá el servidor.
Semana 11	• Análisis y diseño de requisitos: diagrama de la arquitectura cliente-servidor.	**Clase Teórica** **Actividad Innovadora: Clase demostrativa** **Desafío cognitivo:**

		¿Cómo debería diseñarse el sistema para que varios clientes distintos (web, móvil) puedan usar el mismo servidor? **Laboratorio** **Diseño de la arquitectura cliente-servidor** Actividades: ● Elaborar el diagrama de la arquitectura cliente-servidor del proyecto, incluyendo el protocolo de comunicación. ● Diseñar el contrato de la API (rutas, métodos, formato de datos de solicitud y respuesta). ● Validar el diseño con el docente antes de iniciar la implementación.
Semana 12	● Implementación de la aplicación con patrón cliente-servidor.	**Clase Teórica** **Actividad Innovadora: Aprendizaje basado en proyecto** **Desafío cognitivo:** *¿Qué estrategias existen para que un cliente maneje adecuadamente los errores de un servidor que no responde?* **Laboratorio** **Implementación del patrón cliente-servidor** Actividades: ● Codificar el servidor con los endpoints definidos en el diseño (operaciones CRUD básicas). ● Codificar un cliente (aplicación de consola, web o móvil) que consuma los servicios del servidor. ● Probar la comunicación cliente-servidor de extremo a extremo y documentar los resultados. **Evaluación: Parcial No. 2**

ESTRATEGIAS DE APRENDIZAJE (las que utilizará el estudiante)	ACTIVIDADES ACADÉMICAS DE INVESTIGACIÓN Y EXTENSIÓN
● Lluvia de ideas sobre los retos de las arquitecturas cliente-servidor. ● Talleres para la especificación de requisitos del sistema cliente-servidor. ● Clases demostrativas del diseño de contratos de API. ● Aprendizaje basado en proyecto durante la implementación del patrón.	● Investigación sobre variantes del patrón cliente-servidor (dos capas, tres capas, multicapa) y sus aplicaciones actuales. **De extensión:** ● Documentación de la API construida, publicada como referencia para futuros estudiantes del curso.
TÉCNICA O ESTRATEGIA DE EVALUACIÓN	**RECURSOS Y TECNOLOGÍA APLICADA EN EL PROCESO DE ENSEÑANZA Y APRENDIZAJE**
● Documento de requerimientos del escenario cliente-servidor. ● Diagrama de arquitectura y contrato de API evaluados por el docente. ● Evaluación práctica de la implementación funcional del patrón cliente-servidor.	**Documentación oficial:** Documentación oficial del framework de servidor seleccionado (Express/Flask/Spring, entre otros). Documentación del protocolo HTTP/REST. **Software y herramientas:** IDE de desarrollo.

• Parcial No. 2 sobre patrón de capas y patrón cliente-servidor.	Cliente de pruebas de API (ej. Postman). Motor de base de datos. **Recursos didácticos:** Diagramas de arquitectura cliente-servidor de referencia. Plantilla de contrato de API. Plataforma virtual institucional.

FUENTES BIBLIOGRÁFICAS

Huaman, W. C. (2018). Los 10 patrones comunes de arquitectura de software. Medium.
Conesa Caralt, J. (2010). Introducción a .NET. Editorial UOC.

PLANIFICACIÓN DIDÁCTICA

EJE TEMÁTICO	N° 4	Patrón Modelo/Vista/Controlador
COMPETENCIAS GENÉRICAS	• Capacidad de aplicar los conocimientos en la práctica. • Capacidad creativa. • Capacidad de trabajo en equipo.	
COMPETENCIAS ESPECÍFICAS	• Aplica el software requerido para el desarrollo de patrones de diseño arquitecturales. • Implementa patrones de diseño arquitecturales.	
FECHA	**CONTENIDOS**	**ACTIVIDADES ACADÉMICAS INNOVADORAS APLICADAS POR EL DOCENTE**
9 Noviembre – 4 diciembre. Semana 13	• Requerimientos del escenario: definición de una aplicación web con patrón MVC.	**Clase Teórica** **Actividad Innovadora: Taller de diseño** **Desafío cognitivo:** *¿Por qué separar la lógica de presentación (vista) de la lógica de negocio (modelo) facilita el mantenimiento de una aplicación?* **Laboratorio** **Definición del escenario MVC** Actividades: • Analizar el caso de estudio final del curso: una aplicación web que integrará los patrones vistos en el semestre, con MVC como patrón principal. • Identificar los modelos, vistas y controladores que tendrá la aplicación. • Elaborar el bosquejo (wireframe) de las principales pantallas de la aplicación.
Semana 14	• Requisitos del software para la aplicación con patrón MVC.	**Clase Teórica** **Actividad Innovadora: Resolución de problemas** **Desafío cognitivo:** *¿Qué ocurre si un controlador empieza a contener lógica que debería estar en el modelo? ¿Qué problemas genera a futuro?* **Laboratorio** **Especificación de requisitos de la aplicación MVC** Actividades: • Redactar los requisitos funcionales y no funcionales de la aplicación web del proyecto final. • Seleccionar el framework MVC a utilizar (ej. Django, Laravel, Spring MVC, ASP.NET MVC).

		• Definir las rutas (routes) que conectarán las vistas con los controladores.
Semana 15	• Análisis y diseño de requisitos: diagrama del patrón modelo-vista-controlador.	**Clase Teórica** **Actividad Innovadora: Clase demostrativa** **Desafío cognitivo:** *¿Cómo se comunican entre sí el modelo, la vista y el controlador sin que existan dependencias circulares?* **Laboratorio** **Diseño de la arquitectura MVC** Actividades: • Elaborar el diagrama de la arquitectura MVC del proyecto final, mostrando el flujo de datos entre modelo, vista y controlador. • Diseñar la estructura de carpetas del proyecto según las convenciones del framework seleccionado. • Validar el diseño con el docente antes de iniciar la implementación final.
Semana 16	• Implementación de la aplicación con patrón MVC. • Integración de los patrones estudiados durante el semestre.	**Clase Teórica** **Actividad Innovadora: Presentación y defensa del proyecto final** **Desafío cognitivo:** *¿Cómo integrarías los patrones de capas, cliente-servidor y MVC en el diseño de una sola aplicación empresarial?* **Laboratorio** **Implementación y presentación del Proyecto Final** Actividades: • Completar la implementación del modelo, la vista y el controlador de la aplicación web. • Realizar pruebas funcionales de la aplicación completa. • Presentar y defender el proyecto final ante el docente y el grupo, explicando las decisiones arquitectónicas tomadas. **Evaluación: Evaluación final. Examen semestral de integración teórica.**

ESTRATEGIAS DE APRENDIZAJE (las que utilizará el estudiante)	**ACTIVIDADES ACADÉMICAS DE INVESTIGACIÓN Y EXTENSIÓN**
• Talleres de diseño para la definición de escenarios con patrón MVC. • Resolución de problemas sobre la correcta separación de responsabilidades en MVC. • Clases demostrativas del diseño de la arquitectura modelo-vista-controlador. • Aprendizaje basado en proyecto para la implementación y defensa del proyecto final.	• Investigación sobre variantes del patrón MVC (MVP, MVVM) y su comparación con el MVC clásico. **De extensión:** • Presentación del proyecto final desarrollado como muestra de trabajo estudiantil para futuras cohortes del curso.
TÉCNICA O ESTRATEGIA DE EVALUACIÓN	**RECURSOS Y TECNOLOGÍA APLICADA EN EL PROCESO DE ENSEÑANZA Y APRENDIZAJE**
• Documento de requerimientos y wireframes de la aplicación MVC.	**Documentación oficial:** Documentación oficial del framework MVC seleccionado. Guías de patrones de diseño modelo-vista-controlador. **Software y herramientas:**

• Diagrama de arquitectura MVC evaluado por el docente. • Evaluación práctica de la implementación funcional del patrón MVC. • Presentación y defensa del proyecto final. • Examen semestral de integración teórica.	IDE de desarrollo. Framework MVC (Django/Laravel/Spring MVC/ASP.NET MVC, entre otros). Motor de base de datos. **Recursos didácticos:** Diagramas de arquitectura MVC de referencia. Plantilla de wireframes. Plataforma virtual institucional.
FUENTES BIBLIOGRÁFICAS	
Huaman, W. C. (2018). Los 10 patrones comunes de arquitectura de software. Medium. **Blancarte, O. (2020). Introducción a la arquitectura de software.**	

Laboratorio 16 — Presentación del Proyecto Final

Objetivo:

Demostrar el diseño, la implementación y el funcionamiento de la aplicación web desarrollada durante el semestre, integrando los patrones arquitecturales estudiados (capas, cliente-servidor y MVC).

Plantilla de Informe de Laboratorio

Cada informe debe contener:

- Portada
- Objetivo del laboratorio
- Marco teórico breve
- Procedimiento realizado
- Capturas de pantalla / diagramas / evidencias de código
- Resultados obtenidos
- Conclusiones

Guía del Proyecto Final

Proyecto: Aplicación Web con Arquitectura MVC

Requisitos mínimos:

- Aplicación web funcional estructurada con el patrón modelo-vista-controlador.
- Al menos un módulo que refleje el patrón de capas (separación presentación / negocio / datos).
- Comunicación cliente-servidor documentada (endpoints o rutas definidas).
- Persistencia de datos en una base de datos.
- Interfaz de usuario funcional para las operaciones principales del sistema.

Entregables:

1. Documento técnico con las decisiones arquitectónicas
2. Diagramas de arquitectura (capas, cliente-servidor y MVC)
3. Repositorio de código fuente (Git)
4. Aplicación funcional desplegada o ejecutable localmente

Rúbrica de Evaluación del Proyecto

Criterios de evaluación:

Diseño de la arquitectura (capas, cliente-servidor, MVC): 25%
Funcionamiento e integración del sistema: 30%
Calidad del código y separación de responsabilidades: 20%
Documentación técnica: 10%
Presentación y defensa del proyecto: 15%